

Three Layers for a Queueing Library: Surface Syntax, a GSMP Intermediate Representation, and CompetingClocks

WorldTimer research notes

July 11, 2026

Contents

1	What this document is	1
2	Layer 1: the surface language	3
2.1	Vocabulary	3
2.2	A discipline is four functions	3
2.3	Routing kernels	4
2.4	Five models in the syntax	4
3	Layer 2: the intermediate representation	5
3.1	The data structure	5
3.2	Station-to-rule compilation	6
3.3	Worked compilation: the SITA farm	8
3.4	Preemption, processor sharing, and the time-change proposition	9
3.5	Randomness accounting: marks and routing draws	9
3.6	The record and the measurement layer	10
3.7	Model checking on the IR	11
4	Layer 3: CompetingClocks, and the two integrations	11
4.1	The sampler mapping	11
4.2	Embedding in ChronoSim: two event families	12
4.3	Satisfying the ClockGradients contract	13
5	Design findings and open questions	13

1 What this document is

This note makes the three-layer architecture for a queueing library concrete. Layer 1 is the surface language a user types: stations, disciplines, routing. Layer 2 is an intermediate representation (IR): a generalized semi-Markov process (GSMP) written down as a plain data structure, together with a small generic interpreter. Layer 3 is CompetingClocks.jl, which samples the clocks. The document shows the surface syntax for five standard models, then compiles one of them — a

size-based server farm — down to the IR, rule by rule, and shows exactly which `CompetingClocks` and `ChronoSim` calls the interpreter makes.

Four decisions, settled in discussion, are taken as given.

1. **Clock speeds are compiled away.** The GSMP literature gives each clock a state-dependent speed; `CompetingClocks` speaks distributions of firing times, including time-varying hazards. Section 3.4 states the equivalence precisely: a speed is a deterministic time change of the clock’s internal age, and every speed change compiles to a re-enabling with a left-truncated, rescaled distribution. The sampler never sees a speed.
2. **No one-input-queue rule.** Quarton’s structural restriction (every server reads exactly one queue) made every model a state-machine Petri net and priced out fork-join and multiserver jobs. The IR allows synchronization; the surface language exposes it as `fork! / join!`.
3. **Record, then analyze.** Following `ChronoSim`, `CompetingClocks`, and `ClockGradients`, the primary observable is the trajectory record — the ordered (clock key, firing time) sequence. Every queueing statistic is a fold over that record (Section 3.6). Accumulating state inside tokens or queues is not part of the design.
4. **Full-featured target.** The library should support time-varying hazards, static model checking, and derivative estimation through the `ClockGradients` contract. The IR is designed so that the checker and the estimators read the *same* data structure the simulator runs.

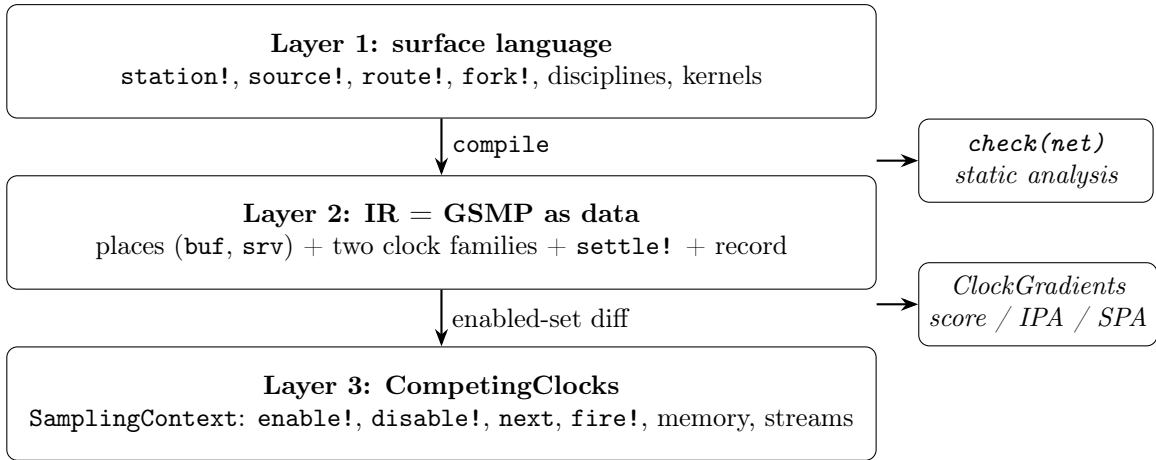


Figure 1: The three layers. The checker and the derivative estimators attach to the IR, not to the surface language and not to the sampler.

The one-sentence contract of each layer:

- *Layer 1 builds a value.* The surface calls `construct` a `QueueNetwork`; nothing executes.
- *Layer 2 is five pure functions over data.* Compilation produces a `QueueGSMP` that satisfies exactly the `ClockGradients` model contract: `initial_state`, `clockkeytype`, `enabled`, `clock_distribution`, `fire`.
- *Layer 3 is driven by diffs.* No rule ever calls the sampler. The interpreter compares `enabled(state)` before and after a firing and issues `enable! / disable!` for the difference — the same pattern as `ClockGradients`’ `ClockWorld` and `ChronoSim`’s reactive engine.

2 Layer 1: the surface language

2.1 Vocabulary

The surface language has six term constructors. Each builds an entry in a `QueueNetwork` value; none has side effects beyond that.

Constructor	Arguments	Meaning
source!	interarrival law, mark sampler	jobs enter here
station!	discipline, servers, service law, capacity	jobs are served here
sink!	—	jobs leave here
fork!	branch list	one job becomes siblings
join!	part count	siblings merge when all arrive
route!	origin, routing kernel	where departures go

A *law* is always a function of $(\theta, \mathbf{t})$ or $(\text{job}, \theta, \mathbf{t})$ returning a `Distributions.UnivariateDistribution`: θ is the global parameter vector (the seam every derivative estimator differentiates through), and $\mathbf{t}$ is the wall-clock enabling time, which is what makes time-varying hazards — a nonhomogeneous arrival process, a repair rate with a shift schedule — expressible with no extra machinery. A *mark* is a named value attached to a job at creation (its size, its class); marks are what disciplines and routing kernels read.

2.2 A discipline is four functions

The design finding that reorganizes Quanton: a service discipline is not a queue subtype. It is a value with four small pure functions, and the buffer is always the same ordered vector.

```

struct Discipline
  insert    # (buf, jobid, state) -> position at which the job is filed
  select    # (buf, state) -> jobid to admit to a free server, or nothing
  preempt   # (state, station) -> jobid in service to evict, or nothing
  memory    # :fresh | :resume | :replay (what an evicted clock remembers)
end

```

The `memory` field is the generalized-stochastic-Petri-net (GSPN) preemption vocabulary, and it maps directly onto ChronoSim’s per-event `memory_policy` declaration: `:fresh` is preemptive-repeat-different (a re-enabled clock starts over), `:resume` banks the enabled age across the disable (ChronoSim implements this by left-shifting the enabling time it hands the sampler), and `:replay` repeats the identical draw (CompetingClocks’ `supports_retained_draw` backends). Standard disciplines are instances:

Discipline	<code>insert</code>	<code>select</code>	<code>preempt</code>	<code>memory</code>
<code>FCFS()</code>	back	front	never	—
<code>LCFS(:resume)</code>	front	front	arriving job evicts	<code>:resume</code>
<code>SIRO()</code>	back	uniform draw	never	—
<code>Priority(by)</code>	ordered by key	front	never	—
<code>Priority(by, preempt=true)</code>	ordered by key	front	lower key in service	<code>:resume</code>
<code>SRPT()</code>	by remaining size	front	larger remainder in service	<code>:resume</code>
<code>ProcessorSharing()</code>	no buffer: every job is in service; see Section 3.4			

Two disciplines need a word. SRPT requires the remaining size of a job in service, which is its size mark minus the clock's enabled age; the IR can read ages because CompetingClocks exposes `enabled_ages` (any `supports_enabled_ages` backend). SIRO's uniform draw is randomness outside any clock; Section 3.5 accounts for it.

2.3 Routing kernels

Both of Quarton's mechanisms — the server's disbursement policy and the queue's hard-coded dispatch — collapse into one concept. A routing kernel is a function `(job, state) -> station id` attached to the *origin* station. State-dependent kernels read buffer lengths or work backlogs from the state; stateful kernels (round robin) declare their cursor, and the cursor lives in the IR state so that record replay reproduces it.

<code>Always(dest)</code>	<i># every departure goes to dest</i>
<code>Probabilistic(dest1 => p1, ...)</code>	<i># Markovian routing (a random draw)</i>
<code>ByMark(f)</code>	<i># f(job) names the destination: SITA</i>
<code>JoinShortestQueue(dests)</code>	<i># reads buffer lengths from state</i>
<code>LeastWorkLeft(dests)</code>	<i># reads remaining-work backlogs from state</i>
<code>RoundRobin(dests)</code>	<i># stateful: cursor is IR state</i>

2.4 Five models in the syntax

M/M/1. The smallest complete program.

```
net = QueueNetwork(param_names = (:lambda, :mu))
source!(net, :arrive; interarrival = ( $\theta$ , t) -> Exponential(inv( $\theta$ [1])))
station!(net, :counter; discipline = FCFS(), servers = 1,
  service = (job,  $\theta$ , t) -> Exponential(inv( $\theta$ [2])))
sink!(net, :done)
route!(net, :arrive, Always(:counter))
route!(net, :counter, Always(:done))
```

SITA farm (size-interval task assignment). Jobs carry a size mark; routing reads it; each service law scales by it. This is the example compiled end-to-end in Section 3.3.

```
net = QueueNetwork(param_names = (:lambda, :mu_short, :mu_long))
source!(net, :arrive;
  interarrival = ( $\theta$ , t) -> Exponential(inv( $\theta$ [1])),
  mark = rng -> (size = rand(rng, LogNormal(0.0, 1.0)),))
station!(net, :short; discipline = FCFS(), servers = 1,
  service = (job,  $\theta$ , t) -> Exponential(job.size * inv( $\theta$ [2])))
station!(net, :long; discipline = FCFS(), servers = 1,
  service = (job,  $\theta$ , t) -> Exponential(job.size * inv( $\theta$ [3])))
sink!(net, :done)
route!(net, :arrive, ByMark(job -> job.size < 2.0 ? :short : :long))
route!(net, :short, Always(:done)); route!(net, :long, Always(:done))
```

Preemptive priority, M/G/1. The discipline carries the preemption rule and the memory policy; the service law is Weibull, so `:resume` is observably different from `:fresh`.

```
station!(net, :cpu;
  discipline = Priority(job -> job.class; preempt = true, memory = :resume),
  servers = 1,
  service = (job,  $\theta$ , t) -> Weibull( $\theta[2]$ ,  $\theta[3]$ ))
```

M/M/1-PS (processor sharing). Only the discipline changes; the compiler does the work (Section 3.4).

```
station!(net, :cpu; discipline = ProcessorSharing(), servers = 1,
  service = (job,  $\theta$ , t) -> Exponential(inv( $\theta[2]$ )))
```

Fork-join. Legal now that the one-input rule is gone. A job splits into two siblings that queue at separate disk stations; the join holds each sibling until its partner arrives.

```
fork!(net, :split; branches = (:disk_a, :disk_b))
station!(net, :disk_a; discipline = FCFS(), servers = 1,
  service = (job,  $\theta$ , t) -> Gamma(2.0, inv( $\theta[2]$ )))
station!(net, :disk_b; discipline = FCFS(), servers = 1,
  service = (job,  $\theta$ , t) -> Gamma(2.0, inv( $\theta[3]$ )))
join!(net, :merge; parts = 2)
route!(net, :disk_a, Always(:merge)); route!(net, :disk_b, Always(:merge))
route!(net, :merge, Always(:done))
```

A closed network needs no new constructs: route the last station back to the first and start `initial_jobs` at chosen stations instead of declaring a source.

3 Layer 2: the intermediate representation

3.1 The data structure

Compilation (`compile(net) -> QueueGSMP`) turns the surface value into pure data plus per-station function fields. Nothing here is a framework: it is a struct you can print, diff, hash, and analyze.

```
struct CompiledStation
  kind::Symbol          # :source | :station | :sink | :fork | :join
  servers::Int
  capacity::Int         # typemax(Int) when unbounded
  overflow::Symbol      # :drop | :block
  discipline::Discipline
  service::Function     # (job,  $\theta$ , t) -> UnivariateDistribution
  routing::Function     # (job, state) -> station index
  mark::Function        # sources only: rng -> NamedTuple
  parts::Int           # joins only: siblings to await
end

struct QueueGSMP
  stations::Vector{CompiledStation}
  names::Dict{Symbol,Int}
end
```

The dynamic state is places. The buffer order *is* the discipline’s data — Shedler’s job stack — so there is no per-discipline container type.

```
struct QueueState
  buf::Vector{Vector{JobId}}      # per station, in discipline order
  srv::Vector{Vector{JobId}}      # per station, the jobs in service
  jobs::Dict{JobId,Job}           # class, marks, birth time index
  pending::Vector{Dict{GroupId,Vector{JobId}}} # joins: siblings so far
  cursor::Vector{Int}             # stateful routing kernels live in state
  next_id::JobId                  # deterministic job identity for replay
end
```

There are exactly two clock families, so the clock key type is one concrete isbits tuple, the way ChronoSim’s `clock_key` flattens an event struct:

```
const ClockKey = Tuple{Symbol,Int32,Int64}
# (:arrival, station, 0)      the source’s next-arrival clock
# (:service, station, jobid) one clock per job in service
```

The enabled-set function is a fixed loop over the state — deterministic order, as the Clock-Gradients contract requires:

```
function enabled(m::QueueGSMP, st::QueueState)
  ks = ClockKey[]
  for s in eachindex(m.stations)
    m.stations[s].kind == :source && push!(ks, (:arrival, Int32(s), 0))
    for j in st.srv[s]
      push!(ks, (:service, Int32(s), j))
    end
  end
  return ks
end
```

3.2 Station-to-rule compilation

The entire dynamic semantics is one `fire` function, one `deposit!` helper, and one normalization function `settle!`. Everything a station means at run time is which branches of these three functions its fields select. No rule touches the sampler; `fire` is a pure state-to-state map (ClockGradients’ contract), and clock lifecycle is implied by the change in `enabled(state)`.

```
function fire(m::QueueGSMP, st::QueueState, key::ClockKey)
  st = fresh_copy(st)          # fire is pure: never mutate input
  fam, s, j = key
  if fam == :arrival
    job = create_job!(st, m, s) # draws marks; Section 3.5
    deposit!(m, st, m.stations[s].routing(job, st), job.id)
  else
    delete_from_service!(st, s, j) # the (:service,s,j) clock vanishes
    deposit!(m, st, m.stations[s].routing(st.jobs[j], st), j)
    settle!(m, st, s)            # the freed slot may admit a job
  end
```

```

end
return st
end

```

```

function deposit!(m::QueueGSMP, st::QueueState, d::Int, j::JobId)
    stn = m.stations[d]
    if stn.kind == :sink
        remove_job!(st, j)                # departure is visible in the record
    elseif stn.kind == :fork
        for b in stn.branches              # siblings share a group id
            deposit!(m, st, b, sibling!(st, j, b))
        end
    elseif stn.kind == :join
        stash_sibling!(st, d, j)
        if all_parts_present(st, d, j)
            file_into_buffer!(st, d, merge_siblings!(st, d, j))
            settle!(m, st, d)
        end
    else                                  # an ordinary station
        if buffer_full(st, stn, d)
            stn.overflow == :drop ? remove_job!(st, j) : block!(st, d, j)
        else
            file_into_buffer!(st, d, j)    # position := discipline.insert
            settle!(m, st, d)
        end
    end
end
end

```

```

function settle!(m::QueueGSMP, st::QueueState, q::Int)
    disc = m.stations[q].discipline
    # 1. Preemption: may the best waiting job evict a job in service?
    while (evict = disc.preempt(st, q)) != nothing
        move_to_buffer!(st, q, evict)    # its clock leaves the enabled set;
    end                                  # disc.memory says what its next
                                         # enabling means (age or no age)

    # 2. Dispatch: fill free slots in discipline order.
    while length(st.srv[q]) < m.stations[q].servers
        j = disc.select(st.buf[q], st)
        j === nothing && break
        move_to_service!(st, q, j)      # its clock joins the enabled set
    end
end
end

```

That is the whole compilation scheme. The correspondence, feature by feature:

Surface feature	Where it lands in the IR
source interarrival law	distribution of the <code>(:arrival, s, 0)</code> clock
service law	distribution of <code>(:service, s, j)</code> clocks
discipline <code>insert/select</code>	<code>file_into_buffer!</code> position; <code>settle!</code> step 2
discipline <code>preempt/memory</code>	<code>settle!</code> step 1; clock memory on re-enable
routing kernel	the destination argument of <code>deposit!</code>
finite capacity, <code>:drop/:block</code>	the <code>buffer_full</code> branch of <code>deposit!</code>
fork, join	the <code>:fork/:join</code> branches of <code>deposit!</code>
sink	job removal; the departure survives in the record

The sub-event ordering worry from Quarton’s `definition.md` — “the danger of separating these steps is that we might exclude a useful model” — is resolved by fiat here: within one firing, the cascade order is `deposit!` at the destination, then `settle!` at the destination, then `settle!` at the origin. The order is part of the IR’s semantics, it is deterministic, and because `fire` is pure it is the *same* order the record fold replays.

3.3 Worked compilation: the SITA farm

The Layer-1 program in Section 2.4 compiles to:

- *Places.* `buf` and `srv` vectors indexed by `{1 arrive, 2 short, 3 long, 4 done}`; only stations 2 and 3 use theirs.
- *Clocks.* `(:arrival, 1, 0)` always enabled; `(:service, 2, j)` for the job `j` occupying `short`’s server; `(:service, 3, j)` likewise for `long`.
- *Laws.* `clock_distribution` dispatches on the key’s family:

```
function clock_distribution(m::QueueGSMP,  $\theta$ , key, st)
    fam, s, j = key
    stn = m.stations[s]
    fam == :arrival && return stn.service(nothing,  $\theta$ , st.time)
    return stn.service(st.jobs[j],  $\theta$ , st.time)    # reads job.size
end
```

One arrival, traced through the interpreter, with the state “`short`’s server busy with job 7, buffers empty”:

1. The sampler reports `(:arrival, 1, 0)` fires at t^* .
2. `fire` runs: `create_job!` makes job 9, draws its mark (`size = 1.3`), and `ByMark` routes it to station 2 (`short`) because $1.3 < 2.0$.
3. `deposit!` files job 9 at the back of `buf[2]` (FCFS `insert`); `settle!(2)` finds no free slot (job 7 holds it) and no preemption (FCFS never preempts). Job 9 waits.
4. The interpreter diffs the enabled sets: before = `{(:arrival, 1, 0), (:service, 2, 7)}`, after adds nothing and removes only the fired arrival clock, which is immediately re-enabled fresh — one `enable!` call with `Exponential(inv( $\theta[1]$ ))` at t^* .

5. Later `(:service, 2, 7)` fires: job 7 departs to `done` (removed), `settle!(2)` admits job 9, and the diff enables `(:service, 2, 9)` with `Exponential(1.3 * inv( $\theta[2]$ ))`.

Every number in that trace is reconstructible from the record $((:arrival, 1, 0), t^*), ((:service, 2, 7), t'), \dots$ by folding **fire** — which is what makes the measurement layer and the estimators work on the record alone.

3.4 Preemption, processor sharing, and the time-change proposition

Terminology accord. Let a clock be enabled at wall time t_e with service requirement $X \sim F$ (hazard h_F), and let it run at a state-dependent speed $r(t) \geq 0$ that is constant between events of the process. Define the internal age $a(t) = \int_{t_e}^t r(u) du$. The clock fires at

$$T = \inf\{t : a(t) \geq X\},$$

and between events its wall-clock hazard is $r(t) h_F(a(t))$. So “a GSMP with clock speeds” and “a GSMP whose clocks carry (possibly time-varying) firing-time distributions, updated at event boundaries” are the same object: the speed is a deterministic time change of the age. Operationally, when an event changes the speed from r_1 to r_2 at time t_1 with banked internal age a_1 , the clock’s remaining wall-clock lifetime is

$$\frac{(X - a_1)}{r_2} \mid X > a_1 \quad \text{i.e.} \quad \text{scale}(\text{LeftTruncated}(F, a_1) - a_1, 1/r_2),$$

a left-truncation and rescale of F — both operations `CompetingClocks` already owns (`distributions/lefttrunc.jl` scaling is `Distributions.jl`). One care point: this compiled re-enabling is anchored at the *current* time because the age bookkeeping has moved into the truncation; returning the original enabling time as well (the usual **reenable** idiom for keeping age) would count the age twice.

Preemption uses the same machinery at speed zero: an evicted job’s clock leaves the enabled set; under **memory** = **:resume** its banked age a_1 is applied as the truncation point when it re-enters service (`ChronoSim`’s **:resume** policy does exactly this by shifting the enabling time); under **:fresh** the bank is discarded.

Processor sharing compiles to: all n jobs at the station sit in **srv**, each with speed k/n (k servers shared by n jobs). Every arrival to or departure from the station changes n , so the interpreter re-enables each resident clock with its truncated, rescaled remainder. Two execution paths support this: `ChronoSim`’s **reenable** seam re-evaluates in-flight clocks natively, and the `ClockGradients` four-argument `clock_distribution(model,  $\theta$ , key, state)` form exists for exactly this mid-flight re-evaluation. The pure `ClockWorld` runner freezes distributions at enabling and is therefore *not* sufficient for PS; the capability table in Section 4.3 records this honestly.

3.5 Randomness accounting: marks and routing draws

A GSMP puts all randomness in clocks. Queueing adds two draws that are not clocks, and the design must say where they live because the likelihood — and therefore the score estimator — depends on it.

- *Job marks* (the size drawn at arrival). The draw happens inside **fire** of the arrival clock. `ChronoSim`’s `CountingRNG` already flags fire-randomness; here we make it lawful instead of

merely flagged. If the mark density g_θ is θ -free (the SITA example: a fixed LogNormal), the likelihood factor is constant and the score is untouched. If marks depend on θ , the trajectory log-likelihood gains a term per arrival:

$$\log L(\theta) = \underbrace{\sum_i \left[\log h_{k_i}(t_i) - \sum_{j \in E(t_{i-1}, t_i]} \int h_j \right]}_{\text{clock part: ChronoSim's trace_likelihood}} + \underbrace{\sum_{\text{arrivals } a} \log g_\theta(\text{mark}_a)}_{\text{mark part: new}}.$$

The record must therefore store marks alongside arrival firings (a *marked* record), which is also what the measurement layer wants.

- *Routing draws* (Probabilistic, SIRO's pick). Same treatment: a discrete density factor, θ -free unless the user parameterizes routing probabilities by θ , in which case the factor $\log p_\theta(\text{choice})$ joins the likelihood. Deterministic kernels (ByMark, JSQ, LWL, RoundRobin with its cursor in state) contribute nothing and keep **fire** a deterministic function of (**state**, **key**, **marks**) — the property the record fold and the pathwise estimators prefer.

This gives a crisp rule the checker can enforce: *every random draw in the system is either a clock, a recorded mark with a density, or a recorded choice with a probability mass*. Nothing else may touch an RNG.

3.6 The record and the measurement layer

The primary output of a run is the marked record

$$\mathcal{R} = ((k_1, t_1, m_1), (k_2, t_2, m_2), \dots), \quad k_i \in \text{ClockKey}, m_i = \text{marks if } k_i \text{ is an arrival},$$

plus the horizon — ChronoSim's **MinimalRecord** with the mark column added. Because **fire** is pure and deterministic given marks, the state at any index is a fold, and the OB-1 measurements showed the fold is *cheaper* than re-running the engine. Every statistic in Harchol-Balter's book is a fold over $\mathcal{R}$:

```
lifecycle = job_events(m, rec)
# folds fire over rec once, emitting per-job daters:
#   (jobid, :created,      station, t)
#   (jobid, :entered_buf, station, t)
#   (jobid, :entered_srv, station, t)   # also re-entries after preemption
#   (jobid, :departed,    station, t)

response(lifecycle)      # departed - created, per job
wait(lifecycle, :short)  # entered_srv - entered_buf at one station
number_in_system(rec)    # a counting-process integral, exact between events
utilization(rec, :short)
```

Shedler's passage-time queries become a two-predicate fold, which is his marked-job specification with the record replacing the state augmentation — the record already contains what his 1987 memory budget forced him to compress:

```
passage_times(lifecycle;
  enter = ev -> ev.kind == :entered_buf && ev.station == :split,
  exit  = ev -> ev.kind == :departed    && ev.station == :merge)
```

Streaming observers remain possible (fold incrementally during the run instead of after it) for long runs where storing $\mathcal{R}$ hurts, but they are an execution-policy choice, not a modeling concept — the same fold, run early.

3.7 Model checking on the IR

Because the IR is data, checks are total functions of it, not lint heuristics over user callbacks. `check(net)` runs at compile time:

- C1. *Structure*. Every routing target exists; every station reaches a sink (or the network is declared closed); every fork’s branches reach a common join or sinks; every join’s **parts** matches its fork.
- C2. *Mark typing*. **ByMark**/**SRPT**/**Priority** require the marks they read to be produced by every source whose jobs can reach them — a dataflow check over the routing graph.
- C3. *Capacity*. Finite buffers declare **:drop** or **:block**; blocking chains must be acyclic (a cycle of full blocking buffers is deadlock, detected statically).
- C4. *Stability*. Solve the traffic equations for visit ratios, estimate $\rho_s = \lambda_s \mathbb{E}[S_s]/k_s$ where the laws permit a mean at the given θ , and warn when $\rho_s \geq 1$.
- C5. *Oracle detection*. If every station satisfies the BCMP conditions (FCFS with exponential class-independent service, or PS, or LCFS-preempt-resume, or infinite-server) and routing is state-independent, emit the product-form stationary distribution as a test oracle. Small all-exponential networks additionally get an exact CTMC oracle by state enumeration. This is how the repo’s 4-standard-error test convention gets its exact answers for free.
- C6. *Randomness census*. List every mark and routing draw with its θ -dependence, and report which estimators are valid (the tier table in Section 4.3).

Runtime invariants (debug builds, ChronoSim’s invariant machinery): job conservation off-source/off-sink; **buf** and **srv** disjoint; **enabled(state)** agreeing with the sampler’s enabled set after every firing.

4 Layer 3: CompetingClocks, and the two integrations

4.1 The sampler mapping

The interpreter drives the blessed **SamplingContext** surface. The complete mapping:

IR notion	CompetingClocks realization
clock key	the <code>ClockKey</code> tuple; context built with this key type
clock born (diff: appears)	<code>enable!(ctx, key, dist, te, now)</code>
clock cancelled (diff: leaves)	<code>disable!(ctx, key, now)</code>
winner & time	<code>next(ctx, now); then fire!(ctx, key, t)</code>
<code>:resume</code> memory	enabling-time shift; <code>supports_enabled_ages</code> backends
<code>:replay</code> memory	<code>supports_retained_draw</code> backends
speed change / PS	re-enable with <code>LeftTruncated</code> + rescale (Sec. 3.4)
time-varying hazard	any <code>UnivariateDistribution</code> ; <code>hazard(d, te, t)</code> is generic
SRPT's remaining size	<code>enabled_ages(ctx, now)</code>
record	<code>TrajectoryRecorder</code> / <code>ChronoSim MinimalRecord</code>
reproducibility	keyed streams from one master seed (<code>KeyedStreams</code>)
sampler choice	<code>SamplerBuilder: NextReactionMethod</code> default; <code>FirstToFireMethod(coupling=:carry)</code> when IPA wants the carry coupling

Non-exponential laws are first-class throughout — that is the reason `CompetingClocks` exists — so “time-varying hazards for rates” costs the surface language nothing: the law functions already receive `t`.

4.2 Embedding in ChronoSim: two event families

The IR needs no code generation to run under `ChronoSim`. Two generic `SimEvent` structs cover every network, because all model specificity lives in the `QueueGSMP` value carried by the physical state:

```

struct Arrival <: SimEvent
    station::Int32
end
struct Service <: SimEvent
    station::Int32
    job::Int64
end

precondition(e::Arrival, phys) = true      # a source is always live
precondition(e::Service, phys) = e.job in phys.state.srv[e.station]

enable(e::Arrival, phys, θ, when) =
    (phys.net.stations[e.station].service(nothing, θ, when), when)
enable(e::Service, phys, θ, when) =
    (phys.net.stations[e.station].service(phys.state.jobs[e.job], θ, when), when)

fire!(e::Service, phys, when, rng) =
    apply!(phys, fire(phys.net, phys.state, (:service, e.station, e.job)))

memory_policy(::Type{Service}) = :resume    # per-network, from the disciplines

```

`ChronoSim`'s generators watch the `srv` places and propose `Service` events when jobs enter service; its `reenable` seam implements the PS rescaling; its `RecordMinimal` policy captures the record. The dynamic semantics of the whole queueing library is those two event families plus `settle!` — everything else is data.

4.3 Satisfying the ClockGradients contract

QueueGSMP implements the five-function model contract directly — `initial_state`, `clockkeytype`, `enabled`, `clock_distribution` (the four-argument, state-dependent form), and `fire` — with one honest wrinkle: the contract’s `fire` is pure and deterministic, and arrivals draw marks. The resolution is the marked record: during estimation, `fire` is evaluated *given* the recorded marks, which restores determinism exactly the way the likelihood conditions on them. The estimator files then apply unchanged; what varies by network feature is which estimators are valid, and the checker’s C6 census reports it:

Network feature	simulate	likelihood/score	IPA (carry)	SPA/branching
exponential + FCFS, det. routing	✓	✓	✓	✓
general laws, time-varying hazards	✓	✓	✓	✓
θ -free marks (SITA)	✓	✓	✓	✓
θ -dependent marks	✓	+ mark term	mark pathwise*	open
θ -dependent Probabilistic routing	✓	+ mass term	discrete: no	✓
preempt-:resume	✓	✓	✓	✓
processor sharing (re-evaluation)	✓	✓	needs framework world	needs framework world

*differentiable if the mark is a smooth transform of a θ -free draw — the usual reparameterization condition.

“Needs framework world” marks the `ClockWorld` freeze-at-enabling limitation: PS re-evaluates clocks mid-flight, so the branching estimators must run over the ChronoSim-backed world, which re-enables through the `reenable` seam.

The payoff, in the shape the repo’s tests want: for the SITA farm,

```
rec = simulate(net,  $\theta$ ; horizon = 200.0, seed = 90210)
W( $\theta$ ) = mean(wait(job_events(compile(net), rec), :short))
# score: ForwardDiff over trace_likelihood + mark terms, weighted by W
# IPA:    carry-coupled replay of the record through clock_distribution
paired_estimate(net,  $\theta$ , W; estimators = (:score, :ipa))
```

with C5’s product-form oracle (this farm is two independent M/G/1 queues once the size split is fixed, so the oracle is closed-form) providing the value *and* the derivative to test against.

5 Design findings and open questions

Findings this layering forces, stated as claims a prototype can test:

1. *Two clock families suffice.* Arrival and Service cover sources, stations, forks, joins, and sinks; disciplines and routing never create new clock kinds, only new state-reading functions. If a queueing feature demands a third family, that is a discovery worth its own note.
2. *A discipline is (insert, select, preempt, memory).* Quarton’s queue subtypes and Assignment objects both dissolve into these four functions plus routing kernels.
3. *Speeds compile to truncate-and-rescale re-enablings.* The sampler needs left-truncation, age reads, and per-clock memory — all of which CompetingClocks has — and nothing else.

4. *The marked record is the single trajectory representation* for statistics, likelihoods, and derivatives; the mark column is the one extension queueing forces on `MinimalRecord`.
5. *Checking is total because the model is a value.* The BCMP/CTMC oracle detector (C5) turns the checker into a test-oracle generator, which is the cheapest route to the repo's 4-standard-error test convention.

Open questions, in rough order of how much they threaten the design:

1. *Blocking cascades.* `:block` with transfer blocking can propagate a single departure through several full buffers. The `deposit!/settle!` order handles one hop; a chain needs a worklist, and the determinism argument must be redone for it.
2. *SRPT's replay.* Preemption decisions read clock ages; the record fold can reconstruct ages from enabling times, but the IPA replay must differentiate through the preemption *boundary* — exactly the kind of discrete-decision discontinuity SPA exists for. Which estimator owns SRPT is unresolved.
3. *θ -dependent marks under SPA.* The branching estimators branch on clock order; a mark that shifts a routing decision is a second discontinuity source not covered by the current branchable-world verbs.
4. *Stateful kernels in records.* `RoundRobin`'s cursor is IR state, so replay works; but a user-written stateful kernel that hides state in a closure breaks the fold silently. The surface API should make kernel state declarative (a field, not a closure), and the checker should reject closures it cannot see into.

The next concrete step this note implies: a green-field `src/QueueLayers` prototype that implements `QueueNetwork`, `compile`, `fire/deposit!/settle!`, the record fold, and checks C1–C4, tested on M/M/1 and the SITA farm against C5-style closed-form oracles — with PS and preemption deferred to a second pass that exercises the `reenable` path.